Fpt University

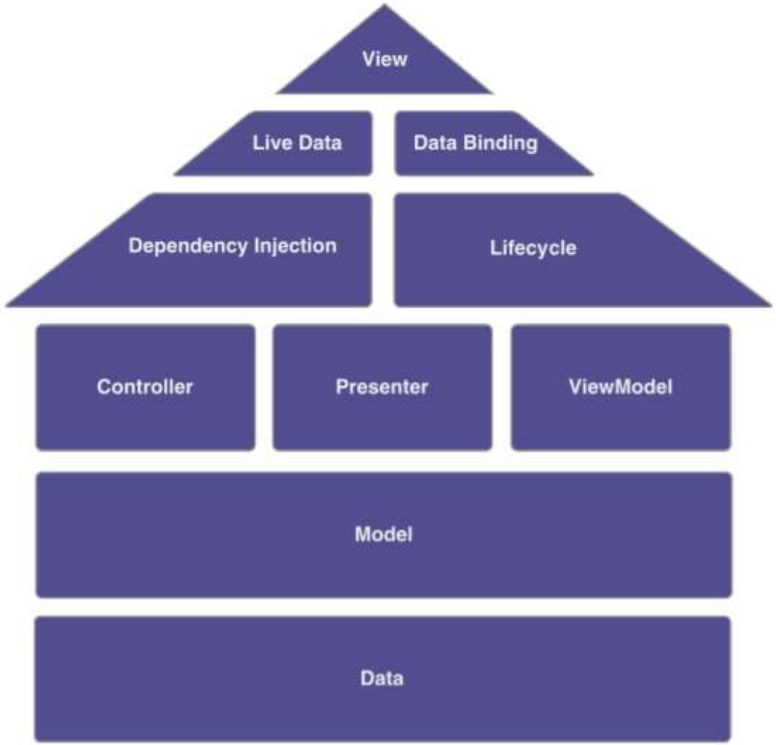
TRƯỜNG ĐẠI HỌC FPT

Android architecture

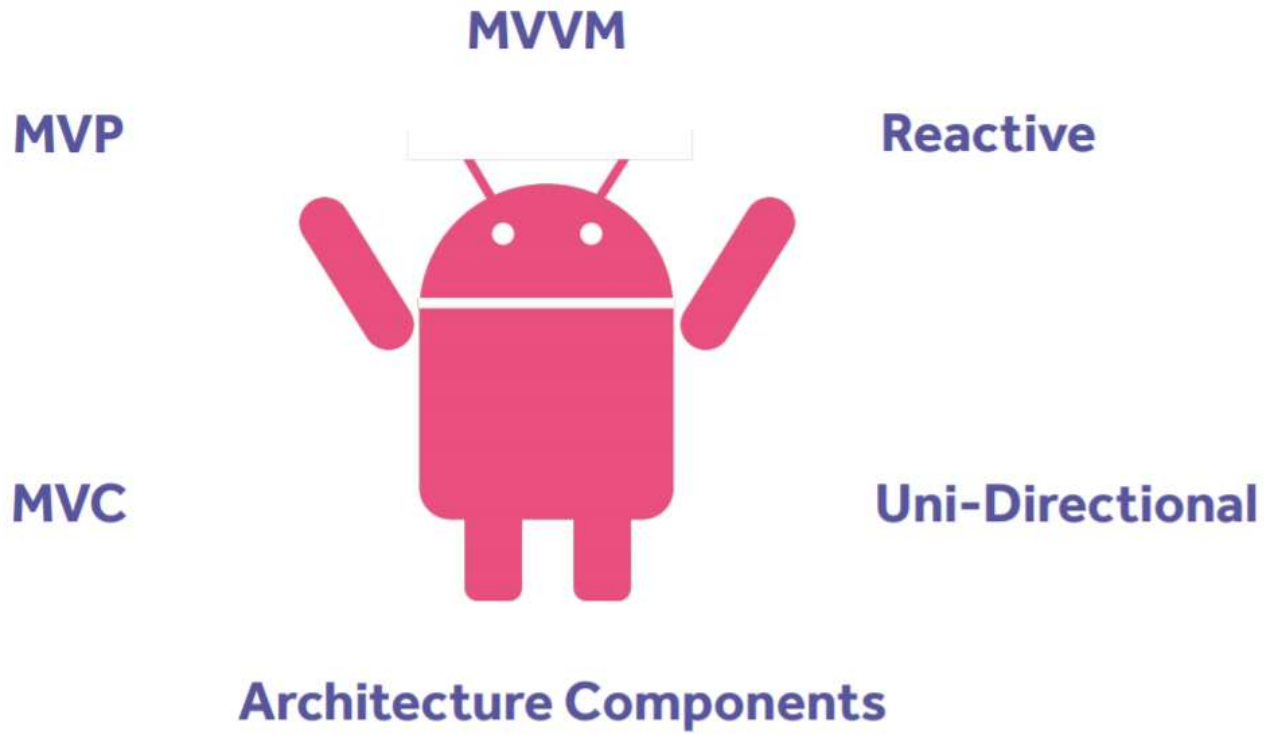
Contents

- **Architecture**
- **MVP**
- **MVVM**

Architecture



Architecture



Sample application

MVC Pattern Architecture

- MVC stands for Model-View-Controller.

MVC components

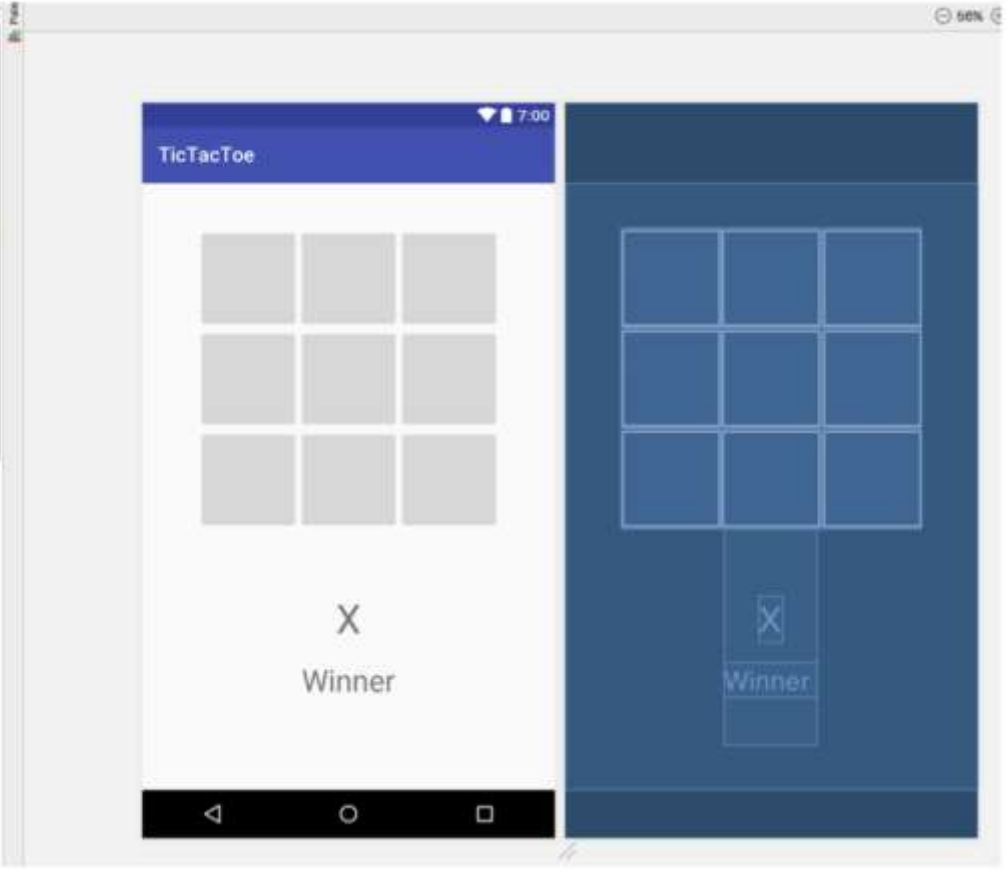
- Model: represents a set of classes that describe the business logic
- View: represents the UI components
- Controller:
  - is responsible to process incoming requests.
  - It receives input from users via the View, then process the user's data with the help of Model and passing the results back to the View.
  - it acts as the coordinator between the View and the Model.
  - There is One-to-Many relationship between Controller and View, means one controller can handle many views

MVC - Model

```java
public class Board {

    private Cell[][] cells = new Cell[3][3];

    private Player winner;
    private GameState state;
    private Player currentTurn;

    private enum GameState { IN_PROGRESS, FINISHED };

    public Board() {...}

    /**
     *  Restart or start a new game, will clear the board and win status
     */
    public void restart() {...}

    /**
     * Mark the current row for the player who's current turn it is.
     * Will perform no-op if the arguments are out of range or if that position is already played.
     * Will also perform a no-op if the game is already over.
     *
     * @param row 0..2
     * @param col 0..2
     * @return the player that moved or null if we did not move anything.
     *
     */
    public Player mark( int row, int col ) {...}
    public Player getWinner() {...}


    private void clearCells() {...}
    private void flipCurrentTurn() {...}
    private boolean isValid(int row, int col ) {...}
    private boolean isOutOfBounds(int idx){...}
    private boolean isCellValueAlreadySet(int row, int col) {...}
    private boolean isWinningMoveByPlayer(Player player, int currentRow, int currentCol) {...}
}

public class Cell {
    private Player value;

    public Player getValue() { return value; }
    public void setValue(Player value) { this.value = value; }
}

public enum Player { X , O }
```

MVC - View

```xml
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/tictactoe"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:paddingBottom="44dp"
    android:paddingLeft="16dp"
    android:paddingRight="16dp"
    android:paddingTop="44dp"
    android:gravity="center_horizontal"
    tools:context="com.acme.tictactoe.controller.TicTacToeActivity">

    <GridLayout
        android:id="@+id/buttonGrid"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:columnCount="3"
        android:rowCount="3">

        <Button
            android:tag="00"
            android:onClick="onCellClicked"
            style="@style/tictactoebutton"
            />

        <Button
            android:tag="01"
            android:onClick="onCellClicked"
            style="@style/tictactoebutton"
            />

        <Button
            android:tag="02"
            android:onClick="onCellClicked"
            style="@style/tictactoebutton"
            />

        <Button
            android:tag="10"
            android:onClick="onCellClicked"
            style="@style/tictactoebutton"
            />

        <Button
            android:tag="11"
            android:onClick="onCellClicked"
            style="@style/tictactoebutton"
            />

        <Button
            android:tag="12"
            android:onClick="onCellClicked"
            style="@style/tictactoebutton"
            />
```



MVC - Controller

```
public class TicTacToeController extends AppCompatActivity {

    private Board model;

    private ViewGroup buttonGrid;
    private View winnerPlayerViewGroup;
    private TextView winnerPlayerLabel;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.tictactoe);
        winnerPlayerLabel = (TextView) findViewById(R.id.winnerPlayerLabel);
        ...
        model = new Board();
    }

    public void onCellClicked(View v) {

        Button button = (Button) v;

        String tag = button.getTag().toString();
        int row = Integer.valueOf(tag.substring(0,1));
        int col = Integer.valueOf(tag.substring(1,2));

        Player playerThatMoved = model.mark(row, col);

        if(playerThatMoved != null) {
            button.setText(playerThatMoved.toString());
            if (model.getWinner() != null) {
                winnerPlayerLabel.setText(playerThatMoved.toString());
                winnerPlayerViewGroup.setVisibility(View.VISIBLE);
            }
        }
    }

    private void reset() {
        winnerPlayerViewGroup.setVisibility(View.GONE);
        winnerPlayerLabel.setText("");
        model.restart();

        for( int i = 0; i < buttonGrid.getChildCount(); i++ ) {
            ((Button) buttonGrid.getChildAt(i)).setText("");
        }
    }
}
```

MVC - Advantages

- Easy to maintain, test and upgrade the multiple system.
- Easy to add new clients just by adding their views and controllers.
- Be able to have development process in parallel for model, view and controller.
- Make the application extensible and scalable.

MVC - Disadvantages

- Requires high skilled experienced professionals
- Requires the significant amount of time to analyze and design.
- This design approach is not suitable for smaller applications.

MVP Pattern

- Model-View-Presenter (MVP) is a variation of the Model-View-Controller (MVC) pattern.
- The primary differentiator of MVP is that the Presenter implements an Observer design of MVC but the basic ideas of MVC remain the same:
  - the model stores the data
  - the view shows a representation of the model
  - the presenter coordinates communications between the layers.

MVP Pattern Architecture

MVP components

- **Model**: The model in MVP is the same as MVC
- **Presenter**:
  - handle user input and use this to manipulate the model data.
  - interactions of the User are first received by the view and then passed to the presenter for interpretation
  - there is One-to-One relationship between View and Presenter
- **View**: When the model is updated, the view also has to be updated to reflect the changes

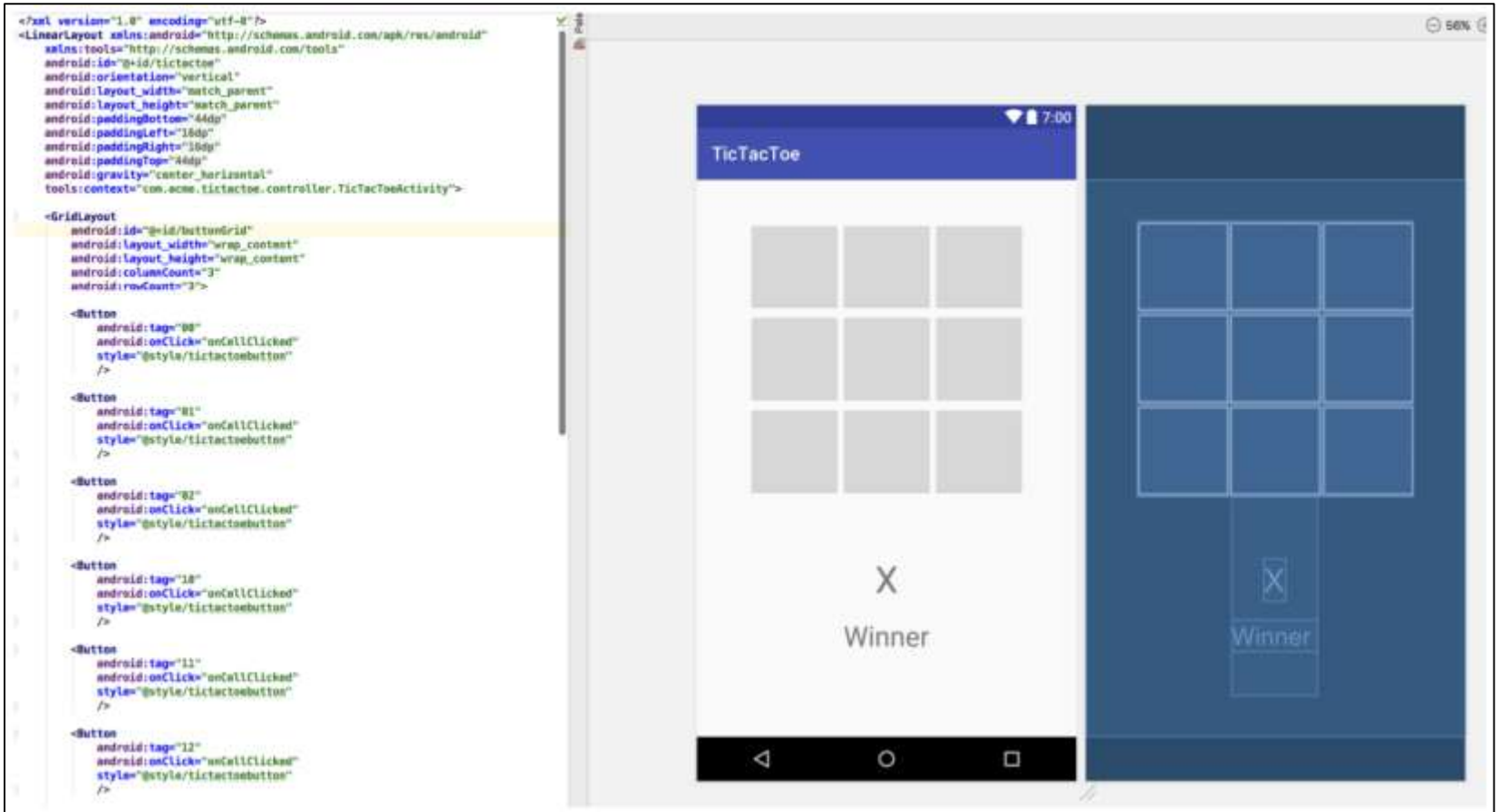
Passive view and Supervising Controller

- The Model-View-Presenter variants: Passive View and Supervising Controller
- In Passive View:
  - the presenter updates the view to reflect changes in the model.
  - The interaction with the model is handled exclusively by the presenter.
  - The view is not aware of changes in the model.
- In Supervising Controller:
  - the view interacts directly with the model to perform simple data-binding that can be defined declaratively, without presenter intervention.
  - The presenter updates the model
  - it manipulates the state of the view only in cases where complex UI logic that cannot be specified declaratively is required.

MVP - Model

```
public class Board {

    private Cell[][] cells = new Cell[3][3];

    private Player winner;
    private GameState state;
    private Player currentTurn;

    private enum GameState { IN_PROGRESS, FINISHED };

    public Board() {...}

    /**
     *  Restart or start a new game, will clear the board and win status
     */
    public void restart() {...}

    /**
     * Mark the current row for the player who's current turn it is.
     * Will perform no-op if the arguments are out of range or if that position is already played.
     * Will also perform a no-op if the game is already over.
     *
     * @param row 0..2
     * @param col 0..2
     * @return the player that moved or null if we did not move anything.
     *
     */
    public Player mark( int row, int col ) {...}
    public Player getWinner() {...}


    private void clearCells() {...}
    private void flipCurrentTurn() {...}
    private boolean isValid(int row, int col ) {...}
    private boolean isOutOfBounds(int idx){...}
    private boolean isCellValueAlreadySet(int row, int col) {...}
    private boolean isWinningMoveByPlayer(Player player, int currentRow, int currentCol) {...}
}

public class Cell {
    private Player value;

    public Player getValue() { return value; }
    public void setValue(Player value) { this.value = value; }
}

public enum Player { X , O }
```

MVP - View


<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:tools="http://schemas.android.com/tools"
android:id="@+id/tictactoe"
android:orientation="vertical"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:paddingBottom="44dp"
android:paddingLeft="16dp"
android:paddingRight="16dp"
android:paddingTop="44dp"
android:gravity="center_horizontal"
tools:context="com.acme.tictactoe.controller.TicTacToeActivity">
<GridLayout
android:id="@+id/buttonGrid"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:columnCount="3"
android:rowCount="3">
<Button
android:tag="00"
android:onClick="onCellClicked"
style="@style/tictactoebutton"
/>
<Button
android:tag="01"
android:onClick="onCellClicked"
style="@style/tictactoebutton"
/>
<Button
android:tag="02"
android:onClick="onCellClicked"
style="@style/tictactoebutton"
/>
<Button
android:tag="10"
android:onClick="onCellClicked"
style="@style/tictactoebutton"
/>
<Button
android:tag="11"
android:onClick="onCellClicked"
style="@style/tictactoebutton"
/>
<Button
android:tag="12"
android:onClick="onCellClicked"
style="@style/tictactoebutton"
/>
56%
7:00
TicTacToe
X
Winner
X
Winner

MVP - View

```java
public interface TicTacToeView {
    void showWinner(String winningPlayerDisplayLabel);
    void clearWinnerDisplay();
    void clearButtons();
    void setButtonText(int row, int col, String text);
}
```

```java
public class TicTacToeActivity extends AppCompatActivity implements TicTacToeView {

    private static String TAG = TicTacToeActivity.class.getName();

    private ViewGroup buttonGrid;
    private View winnerPlayerViewGroup;
    private TextView winnerPlayerLabel;

    TicTacToePresenter presenter = new TicTacToePresenter(this);

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.tictactoe);
        ...
        presenter.onCreate();
    }

    @Override
    protected void onPause() {
        super.onPause();
        presenter.onPause();
    }

    @Override
    protected void onResume() {
        super.onResume();
        presenter.onResume();
    }

    @Override
    protected void onDestroy() {
        super.onDestroy();
        presenter.onDestroy();
    }

    public void onCellClicked(View v) {

        Button button = (Button) v;
        String tag = button.getTag().toString();
        int row = Integer.valueOf(tag.substring(0,1));
        int col = Integer.valueOf(tag.substring(1,2));
        Log.i(TAG, "Click Row: [" + row + "," + col + "]");

        presenter.onButtonSelected(row, col);
    }

    // View Interface Implementation
    public void setButtonText(int row, int col, String text) {...}
    public void clearButtons() {...}
    public void showWinner(String winningPlayerDisplayLabel) {...}
    public void clearWinnerDisplay() {...}
}
```

MVP - Presenter

```
public class TicTacToePresenter extends Presenter {

    private TicTacToeView view;
    private Board model;

    public TicTacToePresenter(TicTacToeView view) {
        this.view = view;
        this.model = new Board();
    }

    @Override
    public void onCreate() {
        model = new Board();
    }

    public void onButtonSelected(int row, int col) {
        Player playerThatMoved = model.mark(row, col);

        if(playerThatMoved != null) {
            view.setButtonText(row, col, playerThatMoved.toString());

            if (model.getWinner() != null) {
                view.showWinner(playerThatMoved.toString());
            }
        }
    }

    public void onResetSelected() {
        view.clearWinnerDisplay();
        view.clearButtons();
        model.restart();
    }
}



public abstract class Presenter {

    public void onCreate() {}
    public void onPause() {}
    public void onResume() {}
    public void onDestroy() {}

}
```

Advantages of MVP

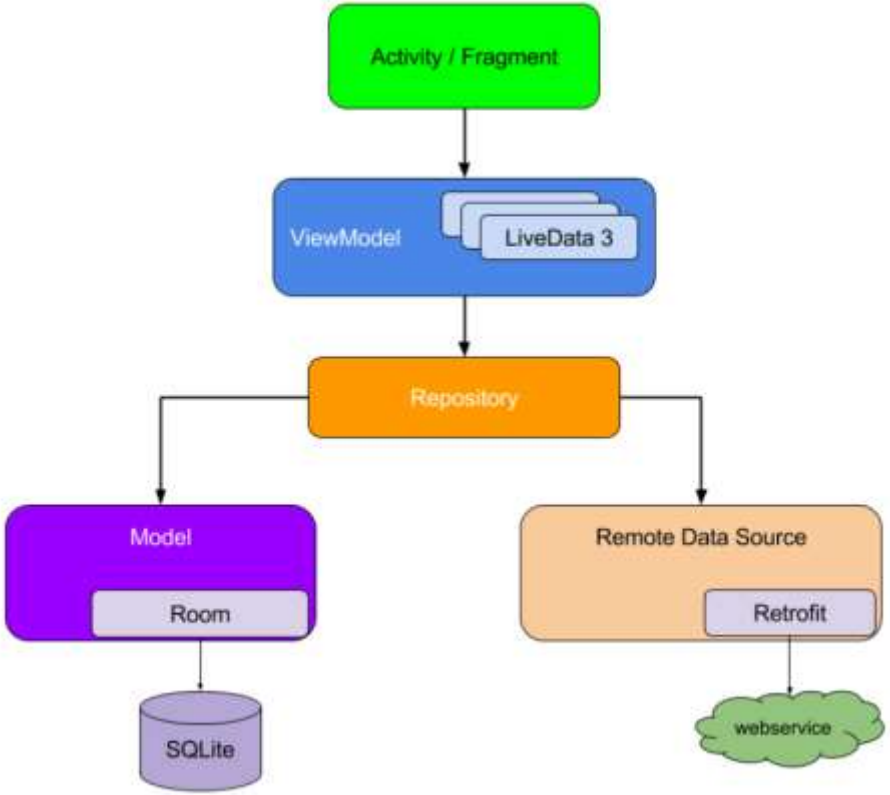
- it clarifies the structure of our user interface code and can make it easier to maintain.
- almost all complex screen logic will reside in the Presenter.
- We have almost no code in our View apart from screen drawing code Model-View-Presenter also makes it theoretically much easier to replace user interface components, whole screens, or even the whole user interface
- It is features can be enhanced separately with more involvement from the customer.
- In addition, unit testing the user interface becomes much easier as we can test the logic by just testing the Presenter.

MVVM Pattern

- The Model-View-ViewModel (MVVM or ViewModel) is a pattern for separating concerns in technologies that use data-binding.
- The MVVM pattern is an adaptation of the MVC and MVP patterns in which the view model provides a data model and behavior to the view but allows the view to declaratively bind to the view model.

MVVM Architecture

MVVM Architecture



MVVM components

- **Model:**
  - It represents the data and the business logic of the Android Application
  - It consists of the business logic - local and remote data source, model classes, repository.
- **View:**
  - It consists of the UI Code(Activity, Fragment), XML
  - It sends the user action to the ViewModel but does not get the response back directly
  - To get the response, it has to subscribe to the observables which ViewModel exposes to it

MVVM components

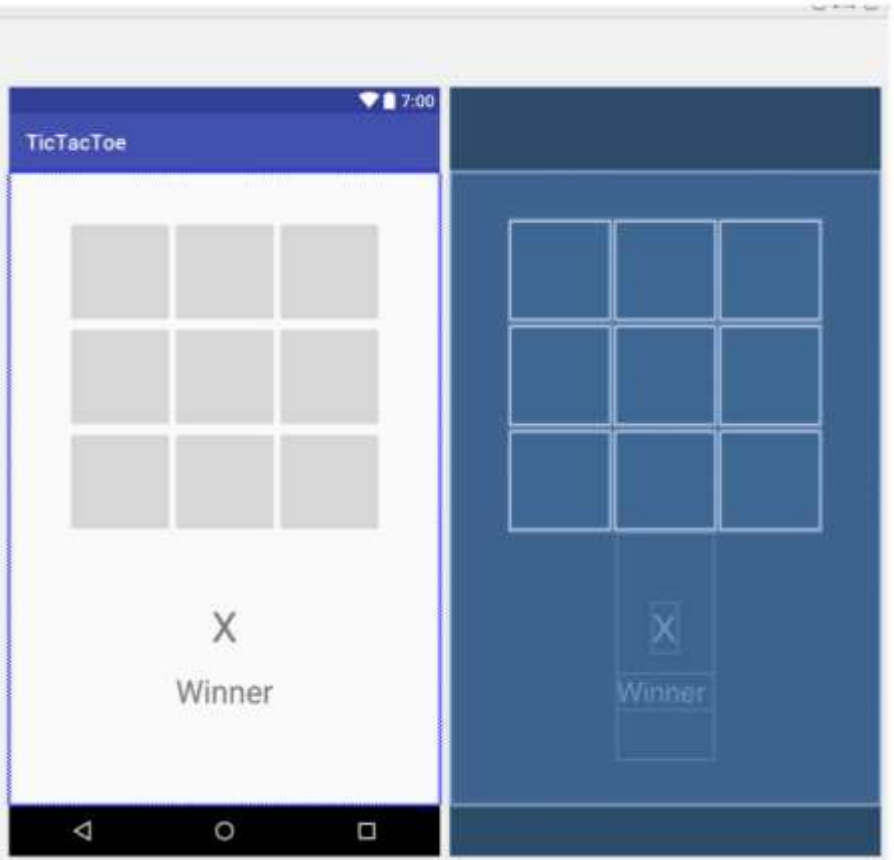
- **ViewModel** :
  - is a bridge between the View and Model
  - It does not have any clue which View has to use it as it does not have a direct reference to the View
  - It interacts with the Model and exposes the observable that can be observed by the View

MVVM - Model

```
public class Board {

    private Cell[][] cells = new Cell[3][3];

    private Player winner;
    private GameState state;
    private Player currentTurn;

    private enum GameState { IN_PROGRESS, FINISHED };

    public Board() {...}

    /**
     *  Restart or start a new game, will clear the board and win status
     */
    public void restart() {...}

    /**
     * Mark the current row for the player who's current turn it is.
     * Will perform no-op if the arguments are out of range or if that position is already played.
     * Will also perform a no-op if the game is already over.
     *
     * @param row 0..2
     * @param col 0..2
     * @return the player that moved or null if we did not move anything.
     *
     */
    public Player mark( int row, int col ) {...}
    public Player getWinner() {...}


    private void clearCells() {...}
    private void flipCurrentTurn() {...}
    private boolean isValid(int row, int col ) {...}
    private boolean isOutOfBounds(int idx){...}
    private boolean isCellValueAlreadySet(int row, int col) {...}
    private boolean isWinningMoveByPlayer(Player player, int currentRow, int currentCol) {...}
}

public class Cell {
    private Player value;

    public Player getValue() { return value; }
    public void setValue(Player value) { this.value = value; }
}

public enum Player { X , O }
```

MVVM - View

```
<layout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <data>
        <import type="android.view.View" />
        <variable name="viewModel" type="com.acme.tictactoe.viewmodel.TicTacToeViewModel" />
    </data>

    <LinearLayout
        android:id="@+id/tictactoe"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:gravity="center_horizontal"
        android:orientation="vertical"
        android:paddingBottom="44dp"
        android:paddingLeft="16dp"
        android:paddingRight="16dp"
        android:paddingTop="44dp"
        tools:context="com.acme.tictactoe.view.TicTacToeActivity">

        <GridLayout
            android:id="@+id/buttonGrid"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:columnCount="3"
            android:rowCount="3">

            <Button
                style="@style/tictactoebutton"
                android:onClick="@(() -> viewModel.onClickedCellAt(0,0))"
                android:text='@(viewModel.cells["00"])' />

            <Button
                style="@style/tictactoebutton"
                android:onClick="@(() -> viewModel.onClickedCellAt(0,1))"
                android:text='@(viewModel.cells["01"])' />

            <Button
                style="@style/tictactoebutton"
                android:onClick="@(() -> viewModel.onClickedCellAt(0,2))"
                android:text='@(viewModel.cells["02"])' />

            <Button
                style="@style/tictactoebutton"
                android:onClick="@(() -> viewModel.onClickedCellAt(1,0))"
                android:text='@(viewModel.cells["10"])' />

            <Button
                style="@style/tictactoebutton"
                android:onClick="@(() -> viewModel.onClickedCellAt(1,1))"
                android:text='@(viewModel.cells["11"])' />

            <Button
                style="@style/tictactoebutton"
                android:onClick="@(() -> viewModel.onClickedCellAt(1,2))"
                android:text='@(viewModel.cells["12"])' />
```



MVVM - View

```
public class TicTacToeActivity extends AppCompatActivity {

    TicTacToeViewModel viewModel = new TicTacToeViewModel();

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        TictactoeBinding binding =
                DataBindingUtil.setContentView(this, R.layout.tictactoe);
        binding.setViewModel(viewModel);
        viewModel.onCreate();
    }

    @Override
    protected void onPause() {
        super.onPause();
        viewModel.onPause();
    }

    @Override
    protected void onResume() {
        super.onResume();
        viewModel.onResume();
    }

    @Override
    protected void onDestroy() {
        super.onDestroy();
        viewModel.onDestroy();
    }
```

MVVM - ViewModel

```
public class TicTacToeViewModel extends ViewModel {

    private Board model;

    public final ObservableArrayMap<String, String> cells = new ObservableArrayMap<>();
    public final ObservableField<String> winner = new ObservableField<>();

    public TicTacToeViewModel() {
        model = new Board();
    }

    public void onResetSelected() {
        model.restart();
        winner.set(null);
        cells.clear();
    }

    public void onClickedCellAt(int row, int col) {
        Player playerThatMoved = model.mark(row, col);
        cells.put(…, …);
        winner.set(…);
    }

}
```

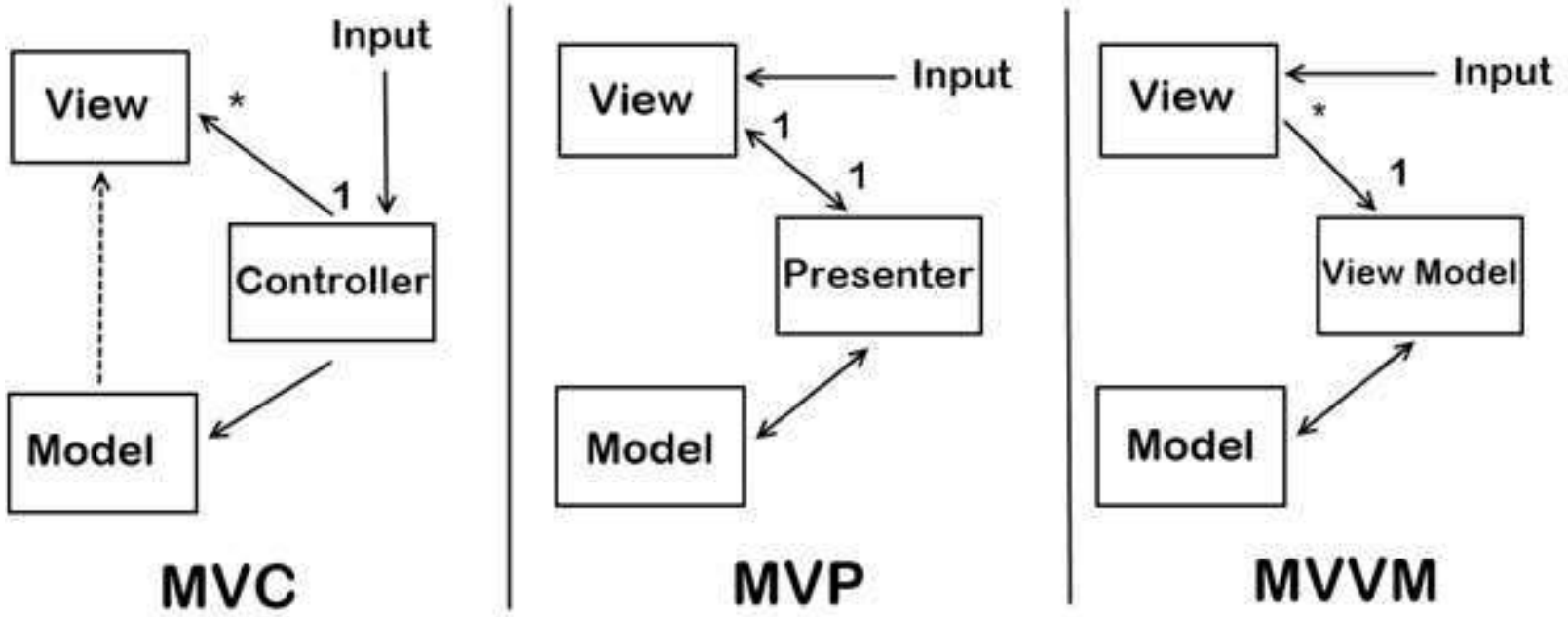
Advantages of MVVM

- Reduces the amount of code in the View's code behind file.
- UI elements can be written in XAML
- Strong Data Binding, which saves a lot of code. No need for manually refresh the view.

Disadvantages of MVVM

- In bigger cases, it can be hard to design the ViewModel up front in order to get the right amount of generality.
- Data-binding for all its wonders is declarative and harder to debug than nice imperative stuff where you just set breakpoints.

Summary


Input
View
*
1
Controller
Model
MVC
View
Input
1
1
Presenter
Model
MVP
View
Input
*
1
View Model
Model
MVVM

References

- https://www.vogella.com/tutorials/AndroidArchitecture/article.html
- https://www.slideshare.net/mudasirqazi00/design-patterns-mvc-mvp-and-mvvm?from_action=save
- https://www.vogella.com/tutorials/AndroidArchitecture/article.html
- https://academy.realm.io/posts/eric-maxwell-mvc-mvp-and-mvvm-on-android/